Lab #11 22th November, 2025

Instructions for Submission

1. File Naming and Submission:
   a. Create a **ZIP file** containing all your solution files.
   b. Name the ZIP file as `lab11_<ERP>.zip`, where `<ERP>` is your ERP.
   c. Ensure that the ZIP file contains:
      i. C++ source files
      ii. No unnecessary files (such as executables, temporary files, or editor configs)
   d. Do not include `TEXT(.txt)` files.
2. Code Neatness:
   a. Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
   b. Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
   c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
3. Code Compilation and Functionality:
   a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
   b. Double-check that the functionality of the program meets the requirements outlined in the task.
4. Additional Instructions:
   a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
   b. Make sure your code adheres to the principles of good software development (e.g., minimal repetition, and no hard-coding).
   c. If you encounter any issues or require clarifications, reach out before the submission deadline.
5. Deadline:
   a. Ensure that you submit the assignment on time. Late submissions will not be accepted.
   b. Double-check that the zip file contains all required files before submitting.
6. Plagiarism Policy:
   a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

In this lecture, we focused on dynamic memory allocation, 1D and 2D arrays, and how to pass arrays to functions in C++. These concepts are essential for handling flexible data structures.

1. Dynamic Memory (1D Arrays)

- Definition: Memory allocated at runtime using the new keyword.
- Syntax:
  ```
  int* arr = new int[n]; // n determined at runtime
  ```
- Key Points:
  - The size of a dynamic array does not need to be known at compile time.
  - Always free memory after use:
    ```
    delete[] arr;
    ```
- Example Use Case: Storing user-defined number of integers, like a list of student marks.

2. Returning a Dynamically Allocated Array

- Concept: A function can create a dynamic array and return its pointer.
- Benefit: Allows the main program to use an array of size determined inside the function.
- Example: Function generating squares or Fibonacci numbers:
  ```
  int* generateSquares(int n);
  ```
- Memory Management: Always remember to **`delete[]`** the returned array to prevent memory leaks.

3. Passing Arrays to Functions

1D Arrays:

- Pass the pointer and size:
  ```
  void printArray(int* arr, int size);
  ```

2D Arrays:

- Static 2D array: Fixed size columns, pass with syntax:
  ```
  void print2D(int arr[][4], int rows);
  ```
- Dynamic 2D array: Use pointer-to-pointer:
  ```
  void print2D(int** arr, int rows, int cols);
  ```
- Important: Dynamic 2D arrays require memory allocation for each row.

4. Dynamic 2D Arrays

- Allocation:
  ```
  int** arr = new int*[rows];
  for(int i = 0; i < rows; i++)
  arr[i] = new int[cols];
  ```
- Deallocation: Free each row, then the array of pointers:
  ```
  for(int i = 0; i < rows; i++) delete[] arr[i];
  delete[] arr;
  ```
- Advantage: Memory is allocated at runtime, flexible row/column size.

5. Summary

- Dynamic memory allows arrays whose size is unknown at compile time.
- Functions can return dynamic arrays for flexible data handling.
- Passing arrays to functions requires understanding of pointers.
- 2D arrays can be static (fixed columns) or dynamic (fully flexible).
- Always free memory to avoid leaks.

Lab Tasks

NOTE: For all tasks, write clear line-by-line comments explaining the process and memory layout of variable.

Task 1: Dynamic Memory [1D Array]

Write a program that:

1. Asks the user how many integers they want to store.
2. Dynamically allocates an array of that size.
3. Allows the user to input values.
4. Prints all the values.
5. Frees the memory.

Requirements

- Use **`new`** and **`delete[]`**
- Use a function **`printArray(int* arr, int size)`**
- Memory must be properly released

Task 2: Return a Dynamically allocated Array

It should:

- Allocate a dynamic array of size n
- Fill it with squares: 0, 1, 4, 9, ...
- Return the pointer to main

Main Program

- Ask the user for n
- Call **`generateSquares(n)`**
- Print all values
- Delete the memory

Function Header:

```
int* generateSquares(int n);
```

Task 3: Passing a Static 2D Array to a Function

Write a program with a fixed 2D array:

```
int arr[3][4];
```

Create functions:

- input2D(int arr[][4], int rows)
- print2D(int arr[][4], int rows)

Steps:

1. Input values using input2D
2. Print the matrix in a grid format

Task 4: Passing a Dynamic 2D Array

Write a program that:

- Asks the user for rows and columns
- Dynamically allocates a 2D array using

```
int** arr = new int*[rows];
for(int i=0; i<rows; i++) arr[i] = new int[cols];
```

- Inputs and prints the matrix
- Frees **all** memory properly
  (delete each row + delete the pointer array)

Required Functions

- `int** create2D(int rows, int cols)`
- `void input2D(int** arr, int rows, int cols)`
- `void print2D(int** arr, int rows, int cols)`
- `void free2D(int** arr, int rows)`

Task 5

Create a program that:

- Dynamically allocates a 2D array
- Finds:
  - Maximum element
  - Minimum element
  - Sum of each row
  - Sum of each column

Print all the results neatly.

Task 6: Observing memory leaks

Write a program that repeatedly allocates memory in a loop without freeing it. Allocate multiple floats in each iteration of the loop. Observe the behavior of the program and the system's memory usage while it is running. Add comments describing the effects of repeated allocations without deallocation. Include observations about changes in memory usage and potential problems caused by not freeing memory.

Task 7: Exploring Dynamic memory with different data types

Create three separate functions, each focusing on a different type of data:

1. Numeric types (such as integers, floats, doubles, or booleans)
2. Character types (like char or string)
3. Pointer types (pointers to any type of your choice)

For each function, allocate memory dynamically for a few variables or arrays, perform simple operations such as assigning values or displaying them, and then free the memory using the appropriate form of **`delete`** or **`delete[]`**. Add comments describing exactly what memory is allocated, how much is allocated for each operation, and when it is deallocated. Ensure that the total allocated memory matches the total deallocated memory for each function.